

CS5481: Data Engineering

Assignment 1

Total marks: 100

Due date: As announced on Canvas.

Instructions

1. Submit either a single PDF containing your written answers together with a code package, or a single Jupyter notebook containing your answers and code. Include the datasets and output files required below.
2. Use Python for the programming exercises. State the Python version, required packages, and commands needed to reproduce your results.
3. For coding questions, explain the main steps of your approach and include the requested outputs. For Question 2, show the regex patterns, the relevant Python calls, and the results for the supplied test cases.

Assignment overview

Question	Topic	Marks
Q1	Web data acquisition	20
Q2	Data cleaning with regular expressions	30
Q3	Data sources and acquisition planning	20
Q4	Data visualization	30

Submission checklist

- **Q1:** Runnable collection code, a CSV or JSON dataset, a request log, one saved listing page and one saved detail page, and a short explanation.
- **Q2:** Ten regex solutions with Python calls and test outputs.
- **Q3:** Written answers and a pipeline diagram or numbered flow.
- **Q4:** Runnable code, generated student data, and the requested visualizations.

The questions are independent. You do not need to use the dataset collected in Q1 to answer the other questions.

Question 1 - Web Data Acquisition

20 marks

An online catalogue contains book information spread across listing pages and individual product pages. Your task is to collect and organize this information using Python.

Source: <https://books.toscrape.com/>

This public website is provided for scraping practice. Its prices and ratings are demonstration data; do not interpret them as real market information.

1. Navigate and collect (5 marks)

Starting from the homepage, follow the **next** links and collect the **first 50 distinct books in listing order**. Visit each book's detail page to obtain the required information. Discover the book URLs and subsequent listing pages from HTML links; do not hard-code a list of 50 URLs or manually copy the records.

2. Extract the required fields (6 marks)

Each record must contain the following fields:

Field	Required content
upc	Product identifier, stored as a string
title	Full book title from the detail page
category	Book category
price_gbp	Price including tax, as a number without the currency symbol
rating	Integer from 1 to 5
stock_count	Available stock as a non-negative integer
source_url	Absolute URL of the book's detail page
collected_at	Collection timestamp in ISO 8601 format, with a timezone

3. Clean and validate (4 marks)

Remove leading/trailing whitespace and unnecessary internal whitespace from text fields. Deduplicate by UPC, keeping the first occurrence. Check that exactly 50 unique records have been collected and that prices, ratings, and stock counts satisfy the specified types and ranges. Report missing or invalid required fields; do not invent values or silently report an incomplete dataset as complete.

4. Document and submit (5 marks)

Save your data in UTF-8 CSV or JSON. Leave at least one second between requests, set a request timeout, and handle failed HTTP requests with a bounded retry policy or an informative error. Keep a request log containing URLs, timestamps, HTTP status codes where available, and error messages for failures. Save at least one listing page and one detail page as raw HTML.

In **150-250 words**, explain your navigation and extraction approach, one data-quality check, and why this task involves both crawling and scraping. Submit your code, data, request log, raw HTML samples, and explanation. If the source becomes unavailable, retain the error evidence and contact the TA through Canvas.

Question 2 - Data Cleaning with Regular Expressions

30 marks; 3 marks for each exercise

Write a regex pattern and the appropriate Python call for each task. Show the results for all supplied test cases. Unless stated otherwise, inputs use ASCII characters and matching is case-sensitive. Use whole-string matching for validation tasks.

1. Alphabetic strings. Check whether a non-empty string contains only alphabetic characters, uppercase or lowercase, without digits or symbols.

Tests: Python, DataScience, Hello123.

2. Hashtag extraction. Extract hashtags, including the #. A hashtag starts with an ASCII letter after #, followed by zero or more letters, digits, or underscores. The # must not be immediately preceded by a letter, digit, underscore, or another #.

Test: Follow #DataEngineering and #CS5481, not #123 or tag#hidden.

3. Domain names. Validate a basic domain name, such as example.com. Labels are separated by dots; non-final labels contain letters, digits, and internal hyphens, and begin/end with a letter or digit. The final label contains at least two letters. Ignore DNS length limits.

Tests: openai.org, invalid@site, my-site.net.

4. Integer extraction. Extract all sequences of digits representing non-negative integers from the following text; return them in order.

Test: He scored 45 goals in 2022 and 10 goals in 2023.

5. Data filenames. Validate a filename ending in lowercase .csv or .json. The non-empty basename begins with a letter, digit, or underscore; later characters may also include hyphens. Paths and spaces are not allowed.

Tests: reviews_2026.csv, book-data.json, image.jpg, /tmp/data.csv.

6. Postal-code format. Validate the format A1A 1A1, where A is an ASCII letter and 1 is a digit, with exactly one space after the third character. Either letter case is accepted; check format only.

Tests: K1A 0B1, 123 456.

7. Matching endpoints. Find strings whose first and last characters are identical. Inputs contain at least two characters and no newline.

Tests: level, stats, world.

8. Whitespace cleaning. Replace each run of spaces, tabs, or newlines with one space, then remove leading/trailing spaces. Use re.sub for the replacement; strip may be used afterward.

Python input: " Data\tEngineering\n is useful. "

9. Date extraction. Use one expression to extract dates in either mm/dd/yyyy or yyyy-mm-dd format. Check digit counts and separators only; calendar validity is not required.

Valid tests: 07/04/2021, 2022-12-31, 01/01/2024.

Invalid tests: 2022/12/31, 13-2020, 07-04-21.

10. IPv4 validation. Validate four decimal integers separated by dots. Each integer must be from 0 to 255; leading zeros are not allowed unless the integer is exactly 0.

Tests: 192.168.1.1, 0.0.0.0, 255.255.255.255, 256.1.1.1, 192.168.01.1, 1.2.3.

Question 3 - Data Sources and Acquisition Planning

20 marks

A university library wants a daily dashboard showing the number of loans by book category. Staff also want access to reader feedback for qualitative review. The following sources are available:

Source	Description
A: Daily CSV export	One loan per row, with fixed columns: loan_id, book_id, and loan_time.
B: Official catalogue API	Returns JSON book objects containing book_id, title, category, and a nested metadata object with optional fields.
C: Public catalogue website	HTML listing and detail pages displaying the same catalogue fields as B, including book_id. Listing pages have next links.
D: Feedback text files	Plain .txt files containing free-form reader comments, without a fixed record schema.

Assume A and B are authorized for this project and contain all information needed for the loan dashboard. Source C contains no additional catalogue fields. No login automation, real API access, or implementation of a database is required for this question.

1. Classify the data (4 marks)

Classify each source as structured, semi-structured, or unstructured **as supplied**. Give one reason for each classification. Distinguish the structure of an HTML document from the free-form text it may contain.

2. Choose acquisition methods (6 marks)

Choose a source and an acquisition method for each need: (i) daily loan records, (ii) book titles and categories, and (iii) reader feedback. Justify each choice. In particular, explain your choice between the catalogue API and scraping the catalogue website.

3. Design a simple pipeline (6 marks)

Draw a diagram or give a numbered flow from acquisition to the dashboard and feedback repository. Include: preservation of raw inputs, validation/cleaning, the key used to combine loan and catalogue data, and the final outputs. Specify **two concrete data-quality checks** and explain what happens to records that fail them. Do not invent category values for unmatched books.

4. Explain crawling and scraping (4 marks)

If the API were unavailable and you had to use source C, distinguish the crawling step from the scraping step in this scenario. Describe two practices that make the collection polite or robust, and explain what each practice achieves.

Submission: Written answers of approximately 400-600 words, plus a pipeline diagram or numbered flow. No code is required for this question.

Question 4 - Data Visualization

(30 marks) Data visualization is an essential tool for exploring and understanding datasets. Common visualization techniques include bar charts, histograms, pie charts, scatter plots, and heatmaps.

1. Suppose we have a dataset of 500 students containing the following attributes: **Student ID** (Integer; 1–500), **Major** (Categorical; Computer Science, Mathematics, Physics), **Gender** (Binary; Male/Female), and **GPA** (Continuous; 0.0–4.0). Which visualization methods would be appropriate to explore the distribution of each attribute and relationships between attributes?
2. Write a Python program to randomly generate 500 student records based on the descriptions above. Visualize the generated data using the visualization techniques you selected in part (a).
3. Compute the number of students in each major and display the results using a bar chart.
4. In recommendation systems, a simple similarity score between user and item embeddings can be calculated using the dot product. Given two matrices $U \in R^{5 \times 8}$ (user embeddings) and $V \in R^{5 \times 8}$ (item embeddings), the similarity score is computed as:

$$\text{Similarity}(U, V) = \text{softmax} \left(\frac{UV^T}{\sqrt{d}} \right),$$

where d is the embedding dimension (8 in this case). Randomly initialize U and V and visualize the similarity scores using a heatmap.